不同搜索树的比较

2026 年 10 月 17 日

Chapter 1

引言

1.1 实验环境

- 编程语言：C
- 编译器：gcc

Chapter 2

数据结构 & 算法描述

本实验当中用到的数据结构有：

- 二叉搜索树 (BST)
- AVL
- 红黑树 (RBT)
- 生长树 (Splay Tree)
- B+ 树 (BPT)

我们将会依次描述其实现方式，给出相应算法的描述（伪代码）。

2.1 二叉搜索树

2.2 AVL

AVL 使用平衡因子 (balance factor)，来维持树的平衡。记树 T 的高度为 H_T，空树高度为-1。每个结点的平衡因子 $bf = H_{T->left} - H_{T->right}$。对于一棵 AVL 树，其每个节点的左右子树的高度差小于等于 1，即 $bf \in \{-1, 0, 1\}$。对树的插入删除都可能会破坏这一性质，因此，我们需要旋转这种操作来维护这种性质。

2.2.1 旋转

旋转，分为左旋和右旋两种操作，由于两者是镜面对称的，因此我们在这里以左旋为例。

```
function LeftRotate(root)
    pivot = root->right
    root->right = pivot->left
    pivot->left = root

    Update the height and balance factor of root and pivot
    return pivot
```

右旋与左旋对称；旋转后返回新的子树根。旋转操作在接下来几种二叉树当中均会使用，后续就不再赘述了。

2.2.2 插入

先根据 BST 的插入方式找到插入位置，然后将新结点插入到该位置。插入后如果破坏了 AVL 树的性质，则需要进行旋转操作来恢复平衡。

```
function AVLInsert(node, key)
    path = empty stack
    current = node

    Search for the insertion position as in a binary search tree
    and push the node passing by onto the stack

    child = CreateNode(key)
    while (path is not empty)
        current = Pop path
        Attach child to current on the side containing key
        Update the height and balance factor of current

        if (current is unbalanced)
            if (left subtree is higher)
                if (key > current->left->key)
                    current->left = LeftRotate(current->left)
                child = RightRotate(current)
            else
                if (key < current->right->key)
                    current->right = RightRotate(current->right)
                child = LeftRotate(current)
        else
            child = current

    return child
```

2.2.3 删除

类似的，先根据 BST 的删除方式找到删除位置，然后将该结点删除。相较于插入，删除之后可能需要更多次的旋转以维持平衡。

```
function AVLDelete(node, key)
    path = empty stack
    current = node

    Search for the deletion position as in a binary search tree
    and push the node passing by onto the stack

    if (current == NULL)
        return node

    if (current has two children)
        Push current and the right-child direction onto path
        successor = current->right
        while (successor->left != NULL)
            Push successor and the left-child direction onto path
            successor = successor->left
        current->key = successor->key
        child = successor->right
        Free successor
    else
        child = current's non-null child, if any
        Free current

    while (path is not empty)
        (current, direction) = Pop path
        Attach child to current in direction
        Update the height and balance factor of current

        if (current is unbalanced)
            if (left subtree is higher)
```

```
            if (current->left balance factor < 0)
                current->left = LeftRotate(current->left)
            child = RightRotate(current)
        else
            if (current->right balance factor > 0)
                current->right = RightRotate(current->right)
            child = LeftRotate(current)
    else
        child = current

return child
```

2.3 红黑树

红黑树对每个结点标记颜色（红/黑），其中结点的颜色需满足以下几条性质：

- 根的颜色为黑色。
- 不能出现连续两个红色结点。
- 每条分路（从根到空结点）的黑色结点数相同。
- 空结点视为黑色。

这个性质保证了最长路径（红黑交替）不超过最短路径（全黑）的两倍。

2.3.1 插入

这里，我们在每个结点当中保存了其父结点的地址，方便回溯、调整。

```
function RBTInsert(T, key)
    Search for the insertion position as in a binary search tree
    If key already exists, return T
    Create a red node new and attach it to its parent
    If new is the root, color it black and return new
    while (new is not the root and its parent is red)
        parent = new->parent
        grandparent = parent->parent
        uncle = the other child of grandparent

        if (uncle is red)
            Color parent and uncle black, and grandparent red
            new = grandparent
        else
            if (parent is the left child of grandparent)
                if (new is the right child of parent)
                    new = parent
                    LeftRotate(parent)
                Color new's parent black and grandparent red
                RightRotate(grandparent)
            else
                if (new is the left child of parent)
                    new = parent
                    RightRotate(parent)
                Color new's parent black and grandparent red
                LeftRotate(grandparent)
            Stop adjusting

    Follow parent pointers to the root and color it black
    return T
```

2.3.2 删除

删除时将实际摘除结点的原颜色记录下来。若其为黑色，则通过调整替代结点及其兄弟结点，恢复各条路径的黑高；空结点视为黑色。

```
function RBTDelete(T, key)
    z = Search for key as in a binary search tree
    if (z == NULL)
        return T

    y = z
    xParent = z->parent
    if (z has at most one child)
        x = z's non-null child, if any
        originalColor = z->color
        Replace z with x; update T if z is the root
        If x is not NULL, update x->parent
    else
        y = findmin(z->right)
        originalColor = y->color
        x = y->right
        if (y->parent == z)
            xParent = y
            If x is not NULL, set x->parent = y
        else
            xParent = y->parent
            Replace y with x
            y->right = z->right
            y->right->parent = y
        Replace z with y; update T if z is the root
        y->left = z->left
        y->left->parent = y
        y->color = z->color

    Free z

    if (originalColor == BLACK)
        // NULL is black; xParent also tracks x when x is NULL
        while (x != T and x is black)
            if (xParent == NULL)
                break
            if (x is the left child of xParent)
                w = xParent->right
            else
                w = xParent->left

            if (w is red)
                Color w black and xParent red
                if (x is the left child)
                    LeftRotate(xParent)
                else
                    RightRotate(xParent)
                Update T if the root changed
                Reassign w

            if (w == NULL)
                x = xParent
                xParent = x->parent
                Continue
            else if (both children of w are black)
                Color w red
                x = xParent
                xParent = x->parent
            else
                if (x is the left child)
                    if (w->right is black)
                        Color w's left child black and w red
                        RightRotate(w)
                        Reassign w
                    Give w xParent's color; color xParent and w's right child black
```

```
                LeftRotate(xParent); update T if the root changed
            else
                if (w->left is black)
                    Color w's right child black and w red
                    LeftRotate(w)
                    Reassign w
                Give w xParent's color; color xParent and w's left child black
                RightRotate(xParent); update T if the root changed
            x = T
            Stop adjusting

    If x is not NULL, color it black
    If T is not NULL, follow parent pointers to the root
    return T
```

2.4 伸展树

2.5 B+ 树

与其他的树不同的是，B+ 树是一种多叉的搜索树，所有的数据存储在叶子结点上，每个叶子结点的高度相同，同时其叶子结点之间通过链表相连，方便范围查询。其多叉的性质使得 B+ 树在磁盘存储上有更好的性能，可以显著减少磁盘 I/O 次数。对于一棵 order 为 M 的 B+ 树，其每个结点最多有 M 个子结点，至少有 $\lceil M/2 \rceil$ 个子结点。在实现方式上，我们使用结点的数组来存储其子结点的指针，同时使用一个数组来存储其关键字。

数据结构体定义如下：

```
typedef struct Bn
{
    int leaf; //判断是否为叶节点，便于分配链表指针
    int* data; //关键字数组
    int num; //当前关键字个数

    struct Bn** children; //子结点指针数组
    struct Bn* next; //叶子结点的链表指针，指向下一个叶子结点
    struct Bn* prev; //叶子结点的链表指针，指向上一个叶子结点
}BNode;
```

插入

```
//树order为M
function insertBPT(T,key)
    path = empty stack
    Search for the insertion position as in a search tree
    and push the node passing by onto the stack

    I = key
    C1 = NULL
    C2 = NULL
    while (True)
        if (path is empty)
            root = newNode()
            root->data[0] = I
            root->children[0] = C1
            root->children[1] = C2
            if (C1 != NULL && C2 != NULL)
                root->leaf = true
            l=else
                root->leaf = false
            T = root
            break

        current = Pop path

```

```
        insert I into current's data array

        if (C1 != NULL && C2 !=NULL)
            insert C1&C2 into current's children array in the corresponding position

        if (current->num > M)
            split current into two node: A & B

            if (current is leaf)
                A->next = B
                B->prev = A
                if (current->prev != NULL)
                    current->prev->next = A
                else
                    update the ptr pointing to the head of the linked list
                B->next = current->next

            free(current)

            I = the first key of B
            C1 = A
            C2 = B

        else
            break

    return T
```

删除

```
```

Chapter 3

实验结果

Chapter 4

总结